

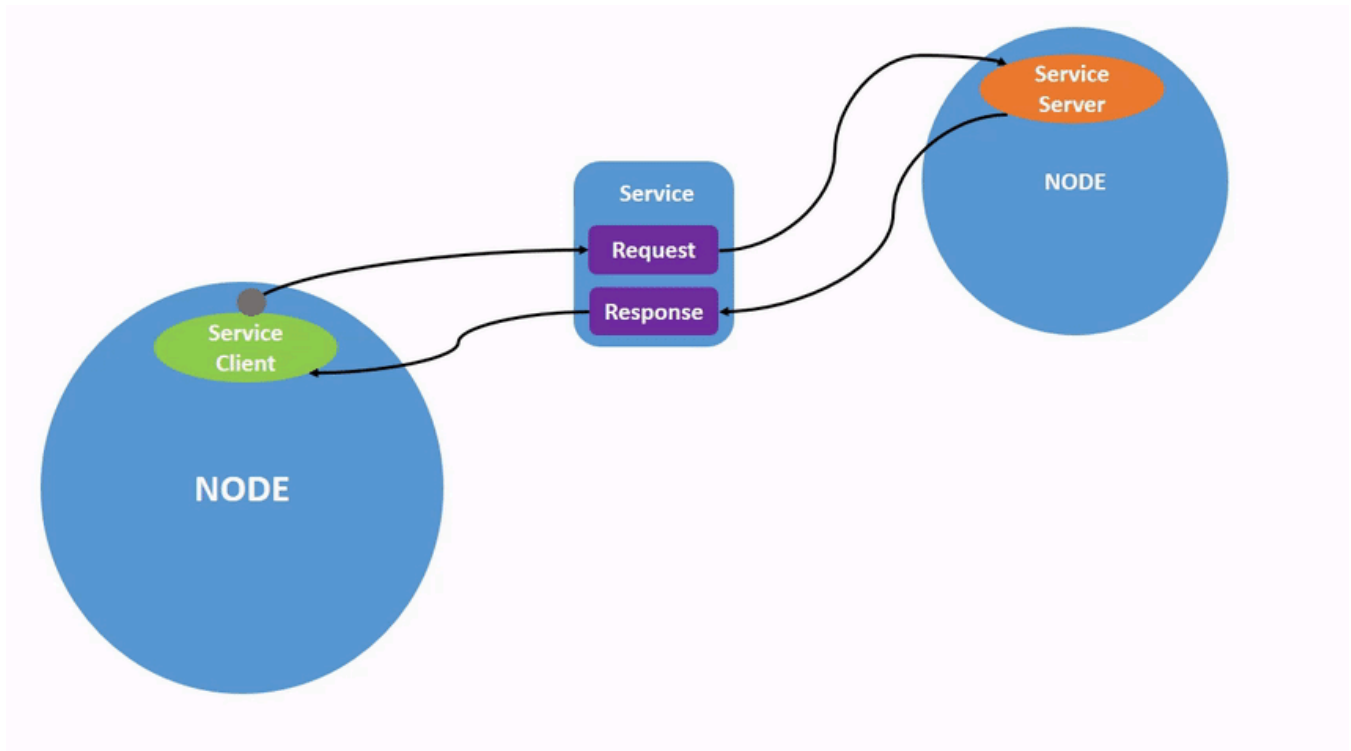
8. ROS2 Service Communication

Note: The accompanying reference video uses the ROS Humble version, for reference only

1. Service Communication Overview

Service communication is a request-response communication model. In communication, the client sends request data to the server, and the server responds with results to the client.

Client/Server model:



From the perspective of service implementation mechanism, this ask-and-answer form is called the Client/Server model, abbreviated as CS model. When the client needs certain data, it sends a request message for a specific service. After the server receives the request, it processes and sends back a response message.

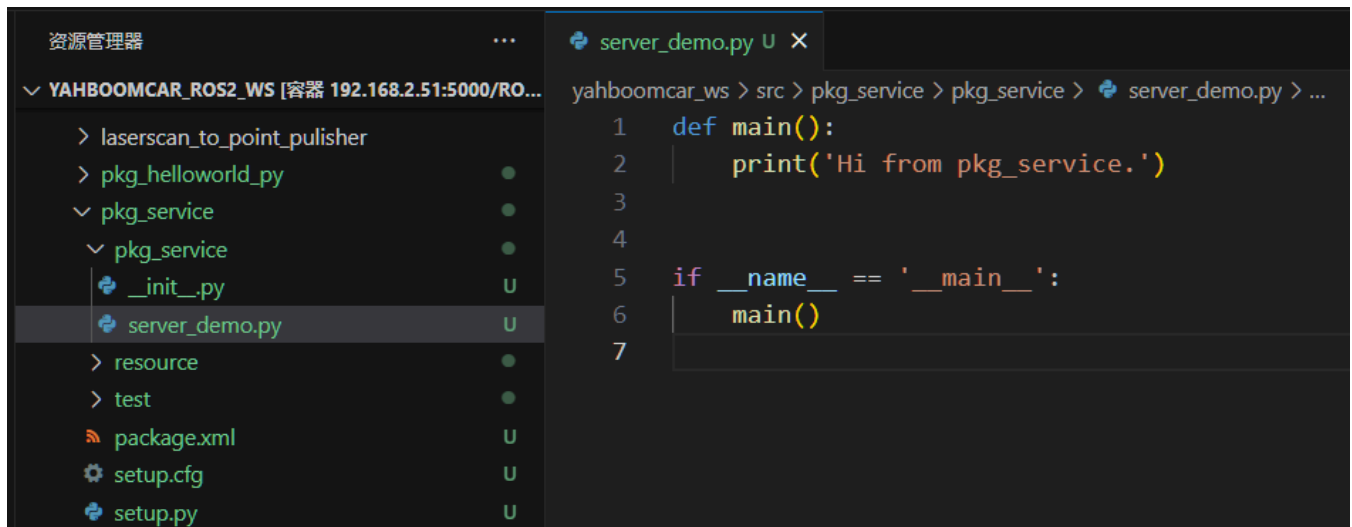
This communication mechanism is also common in real life, such as the web pages we browse. At this time, your computer browser is the client. Through domain names or various operations, it sends requests to the website server. After the server receives the request, it returns the page data to be displayed.

2. Create New Package

- In the workspace src directory:

```
ros2 pkg create pkg_service --build-type ament_python --dependencies rclpy --node-name server_demo
```

After executing the above command, the pkg_service package will be created, along with a server_demo node, and the related configuration files will already be set up.



3. Service Server Implementation

3.1 Create Service Server

Next, edit [server_demo.py] to implement the service server functionality. Add the following code:

```
# Import related library files
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class Service_Server(Node):
    def __init__(self, name):
        super().__init__(name)
        # Create a service server using create_service function with parameters:
        # service data type, service name, service callback function (the service content)
        self.srv = self.create_service(AddTwoInts, '/add_two_ints', self.Add2Ints_callback)
        # The service callback function adds two integers and returns the result
    def Add2Ints_callback(self, request, response):
        response.sum = request.a + request.b
        print("response.sum = ", response.sum)
        return response

def main():
    rclpy.init()
    server_demo = Service_Server("publisher_node")
    rclpy.spin(server_demo)
    server_demo.destroy_node()
    rclpy.shutdown()
```

Key point: Look at the service callback function Add2Ints_callback. Besides self, the parameters are request and response. The request contains the parameters needed for the service, and the response contains the service feedback result. request.a and request.b are the content of the request part, and response.sum is the content of the response part. First, let's look at what the AddTwoInts data type looks like.

- You can use the following command to view:

```
ros2 interface show example_interfaces/srv/AddTwoInts
```

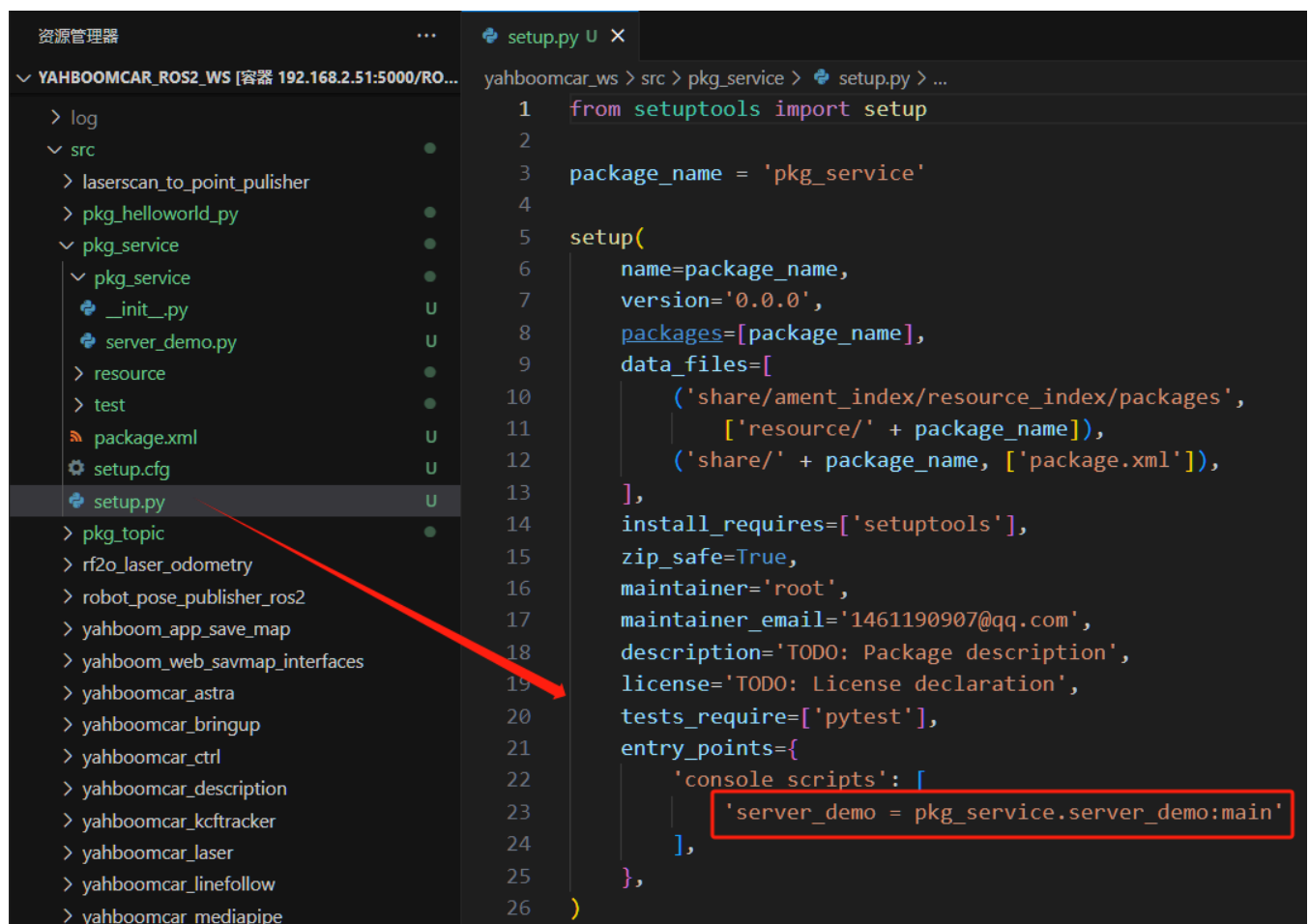
```
root@ubuntu:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 interface show example_interfaces/srv/AddTwoInts
int64 a
int64 b
---
int64 sum
```

"---" divides the data type into two parts. The upper part represents the request, and the lower part represents the response. Each part has its own variables, such as int64 a and int64 b. Therefore, when passing parameters, you need to specify the values for a and b. Similarly, for the feedback result, you need to specify the value for sum.

3.2 Edit Configuration File

- Open setup.py and add to the console_scripts list:

```
'server_demo = pkg_service.server_demo:main',
```

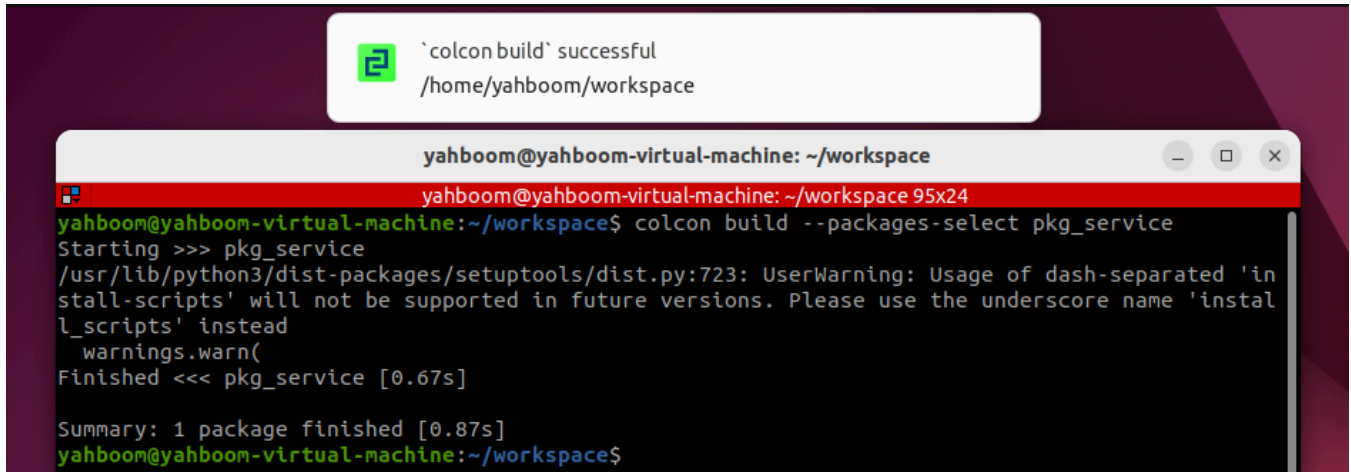


```
1 from setuptools import setup
2
3 package_name = 'pkg_service'
4
5 setup(
6     name=package_name,
7     version='0.0.0',
8     packages=[package_name],
9     data_files=[
10         ('share/ament_index/resource_index/packages',
11          ['resource/' + package_name]),
12         ('share/' + package_name, ['package.xml']),
13     ],
14     install_requires=['setuptools'],
15     zip_safe=True,
16     maintainer='root',
17     maintainer_email='1461190907@qq.com',
18     description='TODO: Package description',
19     license='TODO: License declaration',
20     tests_require=['pytest'],
21     entry_points={
22         'console_scripts': [
23             'server_demo = pkg_service.server_demo:main'
24         ],
25     },
26 )
```

3.3 Build Package

- Build the package:

```
colcon build --packages-select pkg_service
```

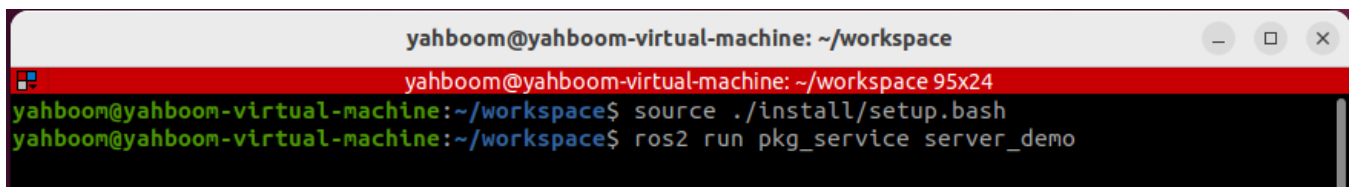


The screenshot shows a terminal window titled 'yahboom@yahboom-virtual-machine: ~/workspace'. A notification bubble at the top left says 'colcon build` successful /home/yahboom/workspace'. The terminal output shows the command 'colcon build --packages-select pkg_service' being executed. The output includes 'Starting >>> pkg_service', a warning about the use of dash-separated names in install scripts, and 'Finished <<< pkg_service [0.67s]'. A summary line states 'Summary: 1 package finished [0.87s]'. The prompt returns to 'yahboom@yahboom-virtual-machine:~/workspace\$'.

3.4 Run Program

- Refresh environment variables and run the node:

```
ros2 run pkg_service server_demo
```



The screenshot shows a terminal window titled 'yahboom@yahboom-virtual-machine: ~/workspace'. The terminal output shows the command 'source ./install/setup.bash' being executed, followed by 'ros2 run pkg_service server_demo'.

After running, since the service is not called, there is no feedback data. You can call the service via command line. First, query what services are currently available. Enter in another terminal:

```
ros2 service list
```

```
root@ubuntu:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 service list
/add_two_ints
/publisher_node/describe_parameters
/publisher_node/get_parameter_types
/publisher_node/get_parameters
/publisher_node/list_parameters
/publisher_node/set_parameters
/publisher_node/set_parameters_atomically
```

/add_two_ints is the service we need to call. Use the following command to call it:

```
ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 1,b: 4}"
```

Here we set a to 1 and b to 4, which means calling the service to calculate the sum of 1 and 4:

```
root@ubuntu:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 1,b: 4}"
requester: making request: example_interfaces.srv.AddTwoInts_Request(a=1, b=4)

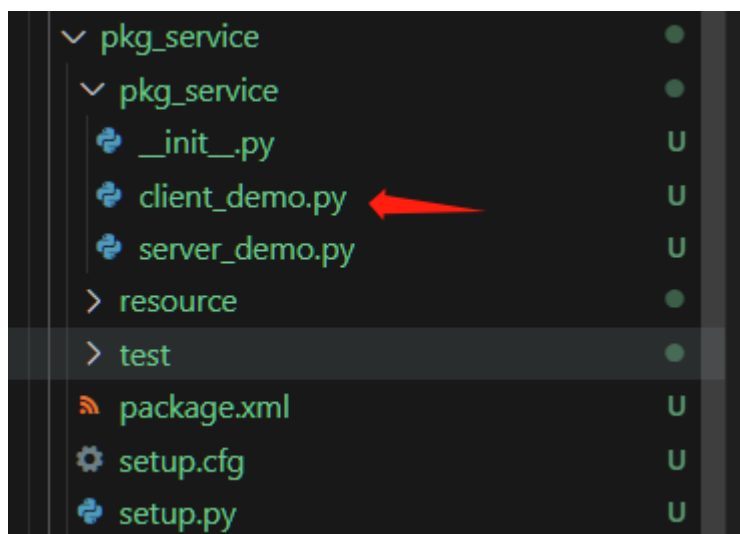
response:
example_interfaces.srv.AddTwoInts_Response(sum=5)
```

As shown in the image, after calling the service, the returned result is 5. The terminal running the service server also prints the returned value.

4. Client Implementation

4.1 Create Client

Create a new file [client_demo.py] in the same directory as [server_demo.py]:



Next, edit [client_demo.py] to implement the client functionality. Add the following code:

```
# Import related libraries
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class Service_Client(Node):
    def __init__(self, name):
        super().__init__(name)
        # Create client using create_client function with parameters:
        # service data type, service topic name
        self.client = self.create_client(AddTwoInts, '/add_two_ints')
        # Loop waiting for server to start successfully
        while not self.client.wait_for_service(timeout_sec=1.0):
            print("service not available, waiting again...")
        # Create service request data object
        self.request = AddTwoInts.Request()

    def send_request(self):
        self.request.a = 10
```

```

        self.request.b = 90
        # Send service request
        self.future = self.client.call_async(self.request)

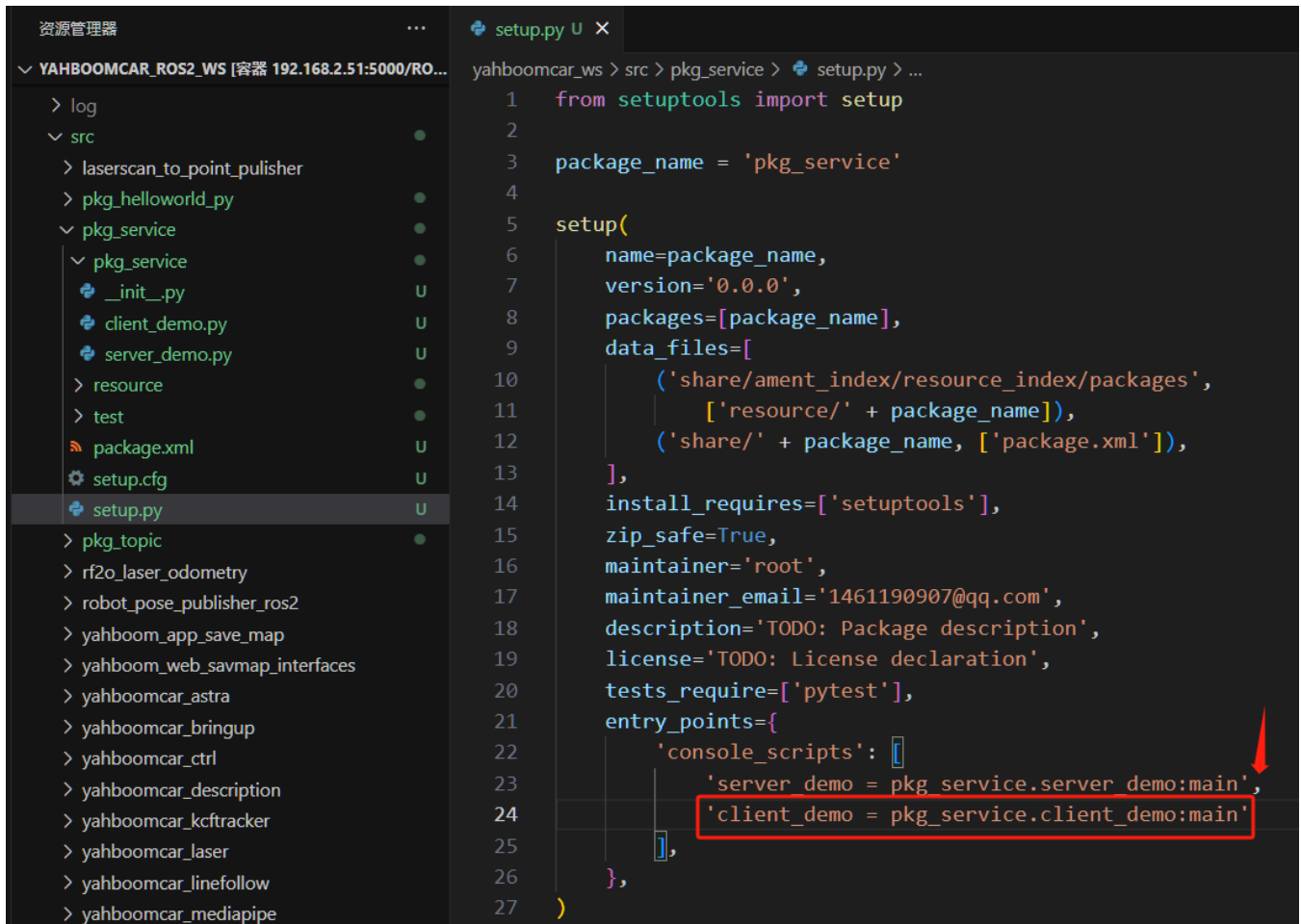
def main():
    rclpy.init() # Node initialization
    service_client = Service_Client("client_node") # Create object
    service_client.send_request() # Send service request
    while rclpy.ok():
        rclpy.spin_once(service_client)
        # Check if data processing is complete
        if service_client.future.done():
            try:
                # Get and print service feedback information
                response = service_client.future.result()
                print("service_client.request.a = ",service_client.request.a)
                print("service_client.request.b = ",service_client.request.b)
                print("Result = ",response.sum)
            except Exception as e:
                service_client.get_logger().info('Service call failed %r' % (e,))
            break
    service_client.destroy_node()
    rclpy.shutdown()

```

4.2 Edit Configuration File

- Open setup.py and add to the console_scripts list:

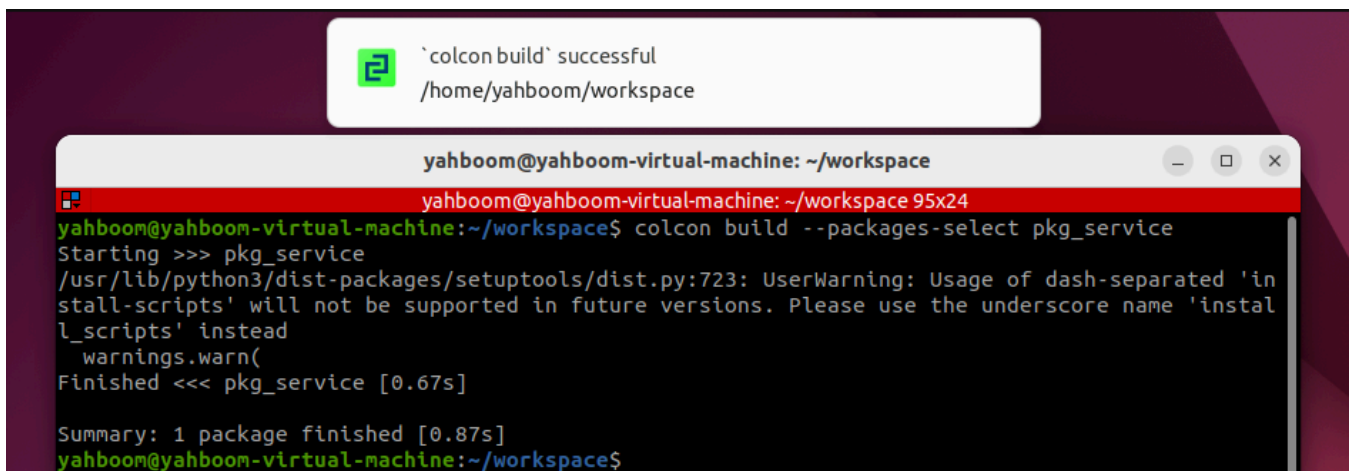
```
'client_demo = pkg_service.client_demo:main'
```



```
1 from setuptools import setup
2
3 package_name = 'pkg_service'
4
5 setup(
6     name=package_name,
7     version='0.0.0',
8     packages=[package_name],
9     data_files=[
10         ('share/ament_index/resource_index/packages',
11          ['resource/' + package_name]),
12         ('share/' + package_name, ['package.xml']),
13     ],
14     install_requires=['setuptools'],
15     zip_safe=True,
16     maintainer='root',
17     maintainer_email='1461190907@qq.com',
18     description='TODO: Package description',
19     license='TODO: License declaration',
20     tests_require=['pytest'],
21     entry_points={
22         'console_scripts': [
23             'server_demo = pkg_service.server_demo:main',
24             'client_demo = pkg_service.client_demo:main'
25         ],
26     },
27 )
```

4.3 Build Package

```
colcon build --packages-select pkg_service
```



```
colcon build` successful
/home/yahboom/workspace

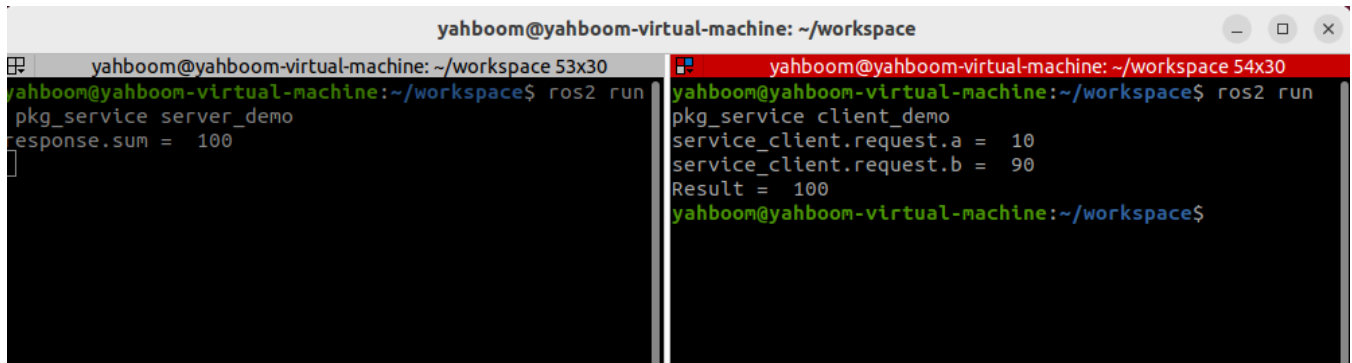
yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine:~/workspace$ colcon build --packages-select pkg_service
Starting >>> pkg_service
/usr/lib/python3/dist-packages/setuptools/dist.py:723: UserWarning: Usage of dash-separated 'install_scripts' will not be supported in future versions. Please use the underscore name 'install_scripts' instead
  warnings.warn(
Finished <<< pkg_service [0.67s]

Summary: 1 package finished [0.87s]
yahboom@yahboom-virtual-machine:~/workspace$
```

4.4 Run Program

- Refresh environment variables and run the node:

```
# Start service server node
ros2 run pkg_service server_demo
# Start client node
ros2 run pkg_service client_demo
```



The image shows two terminal windows side-by-side. The left window, titled 'yahboom@yahboom-virtual-machine: ~/workspace 53x30', shows the command 'ros2 run pkg_service server_demo' being executed, followed by the output 'response.sum = 100'. The right window, titled 'yahboom@yahboom-virtual-machine: ~/workspace 54x30', shows the command 'ros2 run pkg_service client_demo' being executed, followed by the output 'service_client.request.a = 10', 'service_client.request.b = 90', and 'Result = 100'.

```
yahboom@yahboom-virtual-machine: ~/workspace 53x30
yahboom@yahboom-virtual-machine:~/workspace$ ros2 run pkg_service server_demo
response.sum = 100

yahboom@yahboom-virtual-machine: ~/workspace 54x30
yahboom@yahboom-virtual-machine:~/workspace$ ros2 run pkg_service client_demo
service_client.request.a = 10
service_client.request.b = 90
Result = 100
yahboom@yahboom-virtual-machine:~/workspace$
```

Run the service server first, then run the client. The client provides a=10, b=90. The service server calculates the sum, and the result is 100. The result is printed in both terminals.